

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program of basic operations on arrays.

An array is a collection of similar data elements stored in contiguous memory locations. Each element of an array can be accessed using its index, starting from 0. Basic operations on arrays include traversing (displaying elements), insertion, deletion, and searching. Insertion and deletion require shifting elements to maintain the order. Arrays have a fixed size, so operations must be done within the available capacity.

Algorithm for Inserting an Element

Start
Read position (pos) and value
For i = n down to pos+1
 arr[i] = arr[i - 1]
arr[pos] = value
Increase n by 1
Stop

Algorithm for Deleting an Element

Start
Read position (pos)
For i = pos to n - 2
 arr[i] = arr[i + 1]
Decrease n by 1
Stop

Code:

```
#include <iostream>
using namespace std;
int main() {
    int arr[50], n, choice, pos, value, i;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter array elements:\n";
    for (i = 0; i < n; i++) {
        cin >> arr[i];
    }
    do {
        cout << "\n---- ARRAY OPERATIONS ----";
        cout << "\n1. Display";
        cout << "\n2. Insert Element";
        cout << "\n3. Delete Element";
        cout << "\n4. Search Element";
```

```

cout << "\n5. Exit";
cout << "\nEnter your choice: ";
cin >> choice;
switch (choice) {
case 1:
    cout << "Array elements: ";
    for (i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    break;
case 2:
    cout << "Enter position to insert (0-" << n << "): ";
    cin >> pos;
    cout << "Enter value: ";
    cin >> value;
    for (i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = value;
    n++;
    cout << "Element inserted successfully!";
    break;
case 3:
    cout << "Enter position to delete (0-" << n - 1 << "): ";
    cin >> pos;
    for (i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;
    cout << "Element deleted successfully!";
    break;
case 4:
    cout << "Enter value to search: ";
    cin >> value;
    for (i = 0; i < n; i++) {
        if (arr[i] == value) {
            cout << "Element found at index " << i;
            break;
        }
    }
    if (i == n) {
        cout << "Element not found!";
    }
    break;
case 5:
    cout << "Exiting program...";
    break;
default:
    cout << "Invalid choice!";
}
} while (choice != 5);
return 0;

```

}

Output:

Enter number of elements: 5

Enter array elements:

12 14 15 17 20

---- ARRAY OPERATIONS ----

1. Display

2. Insert Element

3. Delete Element

4. Search Element

5. Exit

Enter your choice: 1

Array elements: 12 14 15 17 20

---- ARRAY OPERATIONS ----

1. Display

2. Insert Element

3. Delete Element

4. Search Element

5. Exit

Enter your choice: 3

Enter position to delete (0-4): 2

Element deleted successfully!

---- ARRAY OPERATIONS ----

1. Display

2. Insert Element

3. Delete Element

4. Search Element

5. Exit

Enter your choice: 1

Array elements: 12 14 17 20

---- ARRAY OPERATIONS ----

1. Display

2. Insert Element

3. Delete Element

4. Search Element

5. Exit

Enter your choice: 4

Enter value to search:

14

Element found at index 1

---- ARRAY OPERATIONS ----

1. Display

2. Insert Element

3. Delete Element

4. Search Element

5. Exit

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Illustrate and manipulate multidimensional arrays and matrices.

An array is a collection of elements of the same data type stored in contiguous memory locations.

When arrays have two or more dimensions, they are called multi-dimensional arrays.

The most common type is the 2D array (matrix form), but C++ also supports higher dimensions like 3D, 4D, etc.

Declaration:

```
int arr[rows][cols];
```

Initialization:

```
int arr[2][3] = { {1,2,3}, {4,5,6} };
```

Accessing Elements:

Elements are accessed using two indices → arr[i][j].

Applications

- **Matrices & Algebra** (mathematical computations)
- **Tables** (marksheets, price charts, etc.)
- **Games** (chess board, tic-tac-toe)
- **Images** (2D pixels, 3D RGB images)
- **Graphs and Maps** (pathfinding, adjacency matrices)

Code:

```
#include <iostream>
using namespace std;
int main() {
    int rows, cols;
    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter number of columns: ";
    cin >> cols;
    int arr[10][10];
    // Taking input in 2D array
    cout << "Enter elements of the array:" << endl;
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cin >> arr[i][j];
        }
    }
    // Displaying 2D array
    cout << "\nArray elements are:" << endl;
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
```

```
        cout << arr[i][j] << " ";  
    }  
    cout << endl;  
}  
return 0;  
}
```

Output:

Enter number of rows: 2
Enter number of columns: 3
Enter elements of the array:
10
20
30
40
50
60

Array elements are:
10 20 30
40 50 60

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop programs to demonstrate stack operations (push, pop).

What is Stack?

A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle. i.e insertion and deletion operation are performed at only one end of stack. The first most accessible data in and exceedingly is at top of stack and least accessible data is at the bottom of the stack. Stack has one end and an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack.

1) Push Operation :

- When we insert an element in a stack then the operation is known as a push.
- If we try to insert the element in a stack, and the stack is full, then the **overflow** condition occurs.

Algorithm:

PUSH(stack, top, item)

Step 1: [Is stack full?]

 If(top == max)

 Then display or write “overflow” & exit

Step 2: [increase top by 1]

 Top = top+1

Step 3: [insert the item]

 Stack[top]=item

Step 4: Stop

2) POP Operation :

- When we delete an element from the stack, the operation is known as a pop.
- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the **underflow** condition occurs.

Algorithm

Pop(stack, top, item)

Step 1: [Is stack empty?]

 If(top==-1)

 Then display or write “Underflow” & exit

Step 2: [Delete the top element]

 Item:= Stack[top]

Step 3: [Decrease top by 1]

 Top:=top-1

Step 4: Stop

3) display() - Displays the elements of a Stack

Step 1 [Is stack empty?]

 If(top== -1)

 Then display "**Stack is EMPTY!**"

Step 2 - If it is **NOT EMPTY**, then define a variable 'i' and initialize with top.

 Display **stack[i]** value and decrement **i** value by one .

Step 3 - Repeat above step until **i** value becomes '0'.

Code:

```
#include <iostream>
using namespace std;
int stack[100], n=100, top=-1;
void push(int val) {
    if(top>=n-1)
        cout<<"Stack Overflow"<<endl;
    else {
        top++;
        stack[top]=val;
    }
}
void pop() {
    if(top<=-1)
        cout<<"Stack Underflow"<<endl;
    else {
        cout<<"The popped element is "<< stack[top] <<endl;
        top--;
    }
}
void display() {
    if(top>=0) {
        cout<<"Stack elements are:";
        for(int i=top; i>=0; i--)
            cout<<stack[i]<<" ";
        cout<<endl;
    } else
        cout<<"Stack is empty";
}
int main() {
    int ch, val;
    cout<<"1) Push in stack"<<endl;
    cout<<"2) Pop from stack"<<endl;
    cout<<"3) Display stack"<<endl;
    cout<<"4) Exit"<<endl;
    do {
        cout<<"Enter choice: "<<endl;
        cin>>ch;
        switch(ch) {
            case 1: {
                cout<<"Enter value to be pushed:"<<endl;
                cin>>val;
                push(val);
                break;
            }
        }
    } while(ch>0);
}
```

```

    }
case 2: {
    pop();
    break;
}
case 3: {
    display();
    break;
}
case 4: {
    cout<<"Exit"<<endl;
    break;
}
default: {
    cout<<"Invalid Choice"<<endl;
}
}
}while(ch!=4);
return 0;
}

```

Output:

```

1) Push in stack
2) Pop from stack
3) Display stack
4) Exit
Enter choice:
1
Enter value to be pushed:
12
Enter choice:
1
Enter value to be pushed:
13
Enter choice:
1
Enter value to be pushed:
14
Enter choice:
3
Stack elements are:14 13 12
Enter choice:
2
The popped element is 14
Enter choice:
3
Stack elements are:13 12
Enter choice:

```


Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Modify infix expressions to postfix using polish notation.

➤ **Polish Notation**

- We use mathematical expression for solving the problems. An arithmetic expression contains operands (value), operator and brackets/ delimiters.
- Operators have precedence and associativity. Precedence decides which operation should be performed first.
- If an expression contains two or more operators of equal precedence, then their evaluation is decided by associativity (left to right).
- There are 3 different forms to represent an expression.

1) Infix –

An expression is called the infix expression if the operator appears in the expression in between the operands.

Simply of the form (operand1 operator operand2).

E.g. A+B

2) Postfix –

An expression is called the postfix expression if the operator appears in the expression after the operands.

Simply of the form (operand1 operand2 operator).

E.g. AB+

3) Prefix –

An expression is called the prefix expression if the operator appears in the expression before the operands.

Simply of the form (operator operand1 operand2).

E.g. +AB

Algorithm to Convert Infix to Postfix

1. Start
2. Create an empty stack
3. Scan infix from left to right
4. If operand → add to postfix output
5. If '(' → push to stack
6. If ')' → pop until '(' is removed
7. If operator → pop operators with higher/equal priority, then push current operator
8. After scanning, pop all remaining operators
9. Stop

Algorithm to Convert Infix to Prefix

1. Start
2. Reverse the infix expression
3. Replace '(' with ')' and vice-versa
4. Convert the reversed expression to postfix
5. Reverse the postfix to get prefix
6. Stop

Code:

```
#include <iostream>
#include <stack>
using namespace std;

// Get operator precedence
int prec(char c) {
    if(c == '^') return 3;
    if(c == '*' || c == '/') return 2;
    if(c == '+' || c == '-') return 1;
    return 0;
}

//Display stack contents
void displayStack(stack<char> st) {
    if(st.empty()) {
        cout << "Empty";
        return;
    }
    stack<char> temp;
    while(!st.empty()) {
        temp.push(st.top());
        st.pop();
    }
    while(!temp.empty()) {
        cout << temp.top();
        temp.pop();
        if(!temp.empty()) cout << " ";
    }
}

// Convert infix to postfix with step-by-step display
void infixToPostfix(string s) {
    stack<char> st;
    string result = "";
    for(int i = 0; i < s.length(); i++) {
        char c = s[i];
        cout << " " << c << "\t\t";
        // If operand, add to result
        if((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9')) {
            result += c;
        }
        // If '(', push to stack
        else if(c == '(') {
            st.push(c);
        }
    }
}
```

```

// If ')', pop until '('
else if(c == ')') {
    while(!st.empty() && st.top() != '(') {
        result += st.top();
        st.pop();
    }
    st.pop(); // Remove '('
}
// If operator
else {
    while(!st.empty() && prec(st.top()) >= prec(c)) {
        result += st.top();
        st.pop();
    }
    st.push(c);
}
displayStack(st);
cout << "\t\t" << result << endl;
}
// Pop remaining operators
while(!st.empty()) {
    result += st.top();
    st.pop();
    cout << " -\t\t";
    displayStack(st);
    cout << "\t\t" << result << endl;
}

cout << "\nFINAL POSTFIX EXPRESSION: " << result << endl;
}

int main() {
    string infix;
    cout << "\n(a+b)*c" << endl;
    infixToPostfix("(a+b)*c");
    infixToPostfix(infix);
    return 0;
}

```

Output

```

(a+b)*c
(      (
a      (      a
+      ( +    a
b      ( +    ab
)      Empty   ab+
*      *      ab+
c      *      ab+c
-      Empty   ab+c*|

FINAL POSTFIX EXPRESSION: ab+c*

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Design and implement queue operations.

Queue is a non-primitive linear data structure. It is an ordered list of elements in which addition of new elements can take place from one end called rear and deletion of existing element is done from another end, called as front. This means that elements are removed from a queue in the same order of that in which they were inserted into queue. As the element which inserted first into queue is deleted first from queue so queues are also called First In First Out (FIFO) lists or First Come First Served(FCFS).



1) Insert Element into Queue

Procedure QINSERT(Q,N,F,R, ITEM)

This procedure inserts an ITEM onto rear position of queue. Queue is represented by vector Q containing N elements. F and R are the pointers pointing to the front and rear elements of Q. Initially F and R set to zero.

Step 1: [Is Queue already full?]

If $R \geq N$

Then write("Queue Overflow")

Return

Step 2: [Increase rear by 1]

$R \leftarrow R + 1$

Step 3: [Insert ITEM at rear position]

$Q[R] \leftarrow \text{ITEM}$

Step 4: [Is front pointer property 1]

If $F = 0$

Then $F \leftarrow 1$

Return

2) Delete Element from Queue.

Procedure QDELETE(Q,F,R)

This procedure removes an ITEM from front pointer of queue Q and returns the element. F and R are the pointers pointing to front and rear position of queue resp.

Step 1: [Is Queue empty?]

If $F = 0$

Then write("Queue Underflow")

Return

Step 2: [Assign front element to ITEM]

ITEM $\leftarrow$ Q[F]

Step 3: [Increase F by 1]

F $\leftarrow$ F+1

Step 4: [Finished]

Return

3) display() - Displays the elements of a Queue

- Step 1 - Check whether queue is EMPTY. (front == rear)
- Step 2 - If it is EMPTY, then display "Queue is EMPTY!!!" and terminate the function.
- Step 3 - If it is NOT EMPTY, then define an integer variable 'i' and set 'i = front+1'.
- Step 4 - Display 'queue[i]' value and increment 'i' value by one (i++). Repeat the same until 'i' value reaches to rear (i <= rear)

Code:

```
#include <iostream>
using namespace std;
int queue[100], n = 100, front = -1, rear = -1;
void insertMany(int count) {
    int val;
    for (int i = 0; i < count; i++)
    {
        if (rear == n - 1) {
            cout << "Queue Overflow\n";
            return;
        }
        if (front == -1) front = 0;
        cout << "Enter value " << (i + 1) << ": ";
        cin >> val;
        queue[++rear] = val;
    }
}
void remove() {
    if (front == -1 || front > rear)
        cout << "Queue Underflow\n";
    else
        cout << queue[front++] << " deleted from queue\n";
}
void display() {
    if (front == -1 || front > rear)
        cout << "Queue is empty\n";
    else {
        cout << "Queue elements: ";
        for (int i = front; i <= rear; i++)
            cout << queue[i] << " ";
        cout << endl;
    }
}
int main() {
    int ch, count;
    do {
        cout << "\n1) Insert\n2) Delete\n3) Display\n4) Exit\nEnter choice: ";
        cin >> ch;
        switch (ch) {
```

```

        case 1:
            cout << "How many elements to insert? ";
            cin >> count;
            insertMany(count);
            break;
        case 2: remove(); break;
        case 3: display(); break;
        case 4: cout << "Exiting...\n"; break;
        default: cout << "Invalid choice\n";
    }
} while (ch != 4);
return 0;
}

```

Output:

```

1) Insert
2) Delete
3) Display
4) Exit
Enter choice: 1
How many elements to insert? 5
Enter value 1: 12
Enter value 2: 45
Enter value 3: 85
Enter value 4: 96
Enter value 5: 0

```

```

1) Insert
2) Delete
3) Display
4) Exit
Enter choice: 3
Queue elements: 12 45 85 96 0

```

```

1) Insert
2) Delete
3) Display
4) Exit
Enter choice: 2
12 deleted from queue

```

```

1) Insert
2) Delete
3) Display
4) Exit
Enter choice: 3
Queue elements: 45 85 96 0

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Illustrate the program of the working of deque operations.

A Deque (Double Ended Queue) is a linear data structure in which insertion and deletion can be done from both ends — front as well as rear. It combines features of both queue and stack. Deque is useful in applications like scheduling, sliding window problems, and managing undo–redo operations.

Algorithm for Working of Deque Program

1. Start
2. Create an empty deque 'dq'
3. Insert 10 at the rear of deque
4. Insert 20 at the rear of deque
5. Insert 30 at the front of deque
6. Display all elements of deque
7. Remove one element from the front
8. Remove one element from the rear
9. Display the remaining elements of deque
10. Stop

Code:

```
#include <iostream>
#include <deque>
using namespace std;
int main() {
    deque<int> dq;
    dq.push_back(10);
    dq.push_back(20);
    dq.push_front(30);
    // Print deque elements
    for (int x : dq) cout << x << " ";
    cout << endl;
    // Pop from front and back
    dq.pop_front();
    dq.pop_back();
    // Print deque elements after pop
    for (int x : dq)
        cout << "The queue after Popped elements:" << x << " ";
    return 0;
}
```

Output:

30 10 20
The queue after Popped elements:10

Name: _____ Class: _____

Expt.Title: _____

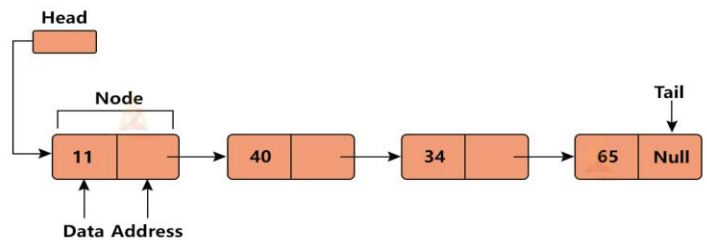
Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Create and demonstrate linear linked list operations.

The term list refers to a linear Collection of data items. This type of data structure is called a linked list. The first node of the singly linked list is called as head of the linked list.

- 1) The first part of node contains the info of the element, and
- 2) The second part contains the address of the next node in the list. It is also called as link field or next pointer field.
- 3) The last node of the singly linked list is called the tail of the linked list and it points to null, which indicates the end of the linked list.



1) Algorithm for Creation of a Node

Step 1: Get the new node using getnode().

newnode = getnode();

Step 2: If the list is empty, assign new node as start.

start = newnode;

Step 3: If the list is not empty, follow the steps given below:

- The next field of the new node is made to point the first node (i.e. start node) in the list by assigning the address of the first node.
- The start pointer is made to point new node by assigning address of new node times.

Step 4: Repeat the above steps 'n' times.

2) Algorithm for Insert Node

Step 1: Get the new node using getnode().

newnode = getnode();

Step 2: If the list is empty, assign new node as start.

Start= newnode;

Step 3: If the list is not empty, follow the steps given below

Newnode → next=start;

Start= newnode;

Step 4: Stop

3) Algorithm for Delete Node

Step 1: Check If list is empty then

Display 'Empty List' message.

Step 2: If the list is not empty, follow the steps given below:

temp = start;

start start->next;

Step 3: Free(temp);

Code:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
void insertStart(Node** head, int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = *head;
    *head = newNode;
    cout << data << " inserted\n";
}
void deleteStart(Node** head) {
    if (*head == NULL) {
        cout << "List empty, nothing to delete\n";
        return;
    }
    Node* temp = *head;
    *head = (*head)->next;
    cout << temp->data << " deleted\n";
    delete temp;
}
void display(Node* node) {
    cout << "Linked List: ";
    while (node) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << "\n";
}
int main() {
    Node* head = NULL;
    insertStart(&head, 100);
    insertStart(&head, 80);
    insertStart(&head, 60);
    display(head);
    deleteStart(&head);
    display(head);
    return 0;
}
```

Output:

```
100 inserted
80 inserted
60 inserted
Linked List: 60 80 100
60 deleted
Linked List: 80 100
```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

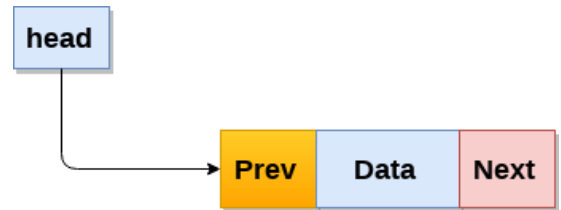
Remarks: _____ Sign: _____

Title: Write a program which shows the use of doubly linked list operations.

A Doubly Linked List (DLL) is a type of linked list in which each node contains three parts:

1. Data field – stores the actual value.
2. Pointer to the next node (next link).
3. Pointer to the previous node (previous link).

Thus, a DLL allows bidirectional traversal (both forward and backward).



Node

Structure of Node

```
struct Node {
    int data;
    Node* prev; // points to previous node
    Node* next; // points to next node
};
```

Code:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* prev;
    Node* next;
};
Node* head = NULL;
// Insert at end
void insert(int val) {
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
    cout << val << " inserted.\n";
}
```

```
void display() {
    Node* temp = head;
    if (!temp) {
        cout << "List is empty.\n";
        return;
    }
    cout << "Doubly Linked List: ";
    while (temp != NULL) {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL\n";
}

int main() {
    insert(10);
    insert(20);
    insert(30);
    display();
    return 0;
}
```

Output:

10 inserted.
20 inserted.
30 inserted.
Doubly Linked List: 10 <-> 20 <-> 30 <-> NULL

Name: _____ Class: _____

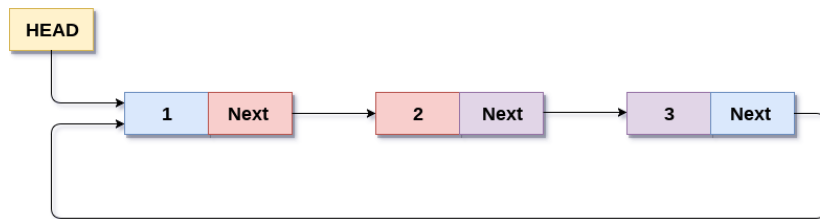
Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop a program of circular linked list operations.

- In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.
- We traverse a circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.



Circular Singly Linked List

Algorithm for Circular Linked List

1. Start
2. Create an empty circular linked list with last = NULL
3. Create a new node with data = 10
4. Since list is empty, point new node to itself and set last = new node
5. Create a new node with data = 20
6. Link new node after last and update last to this new node
7. Create a new node with data = 30
8. Link new node after last and update last to this new node
9. Set temp pointer to the first node (last->next)
10. Traverse the list and print all nodes until temp reaches first node again
11. Stop

Code:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
class CircularLinkedList {
    Node* last;
```

```

public:
    CircularLinkedList() { last = NULL; }
    void insert(int value) {
        Node* newNode = new Node;
        newNode->data = value;

        if (last == NULL) {
            last = newNode;
            last->next = last; // first node points to itself
        } else {
            newNode->next = last->next;
            last->next = newNode;
            last = newNode;
        }
    }
    void display() {
        if (last == NULL) {
            cout << "List is empty\n";
            return;
        }
        Node* temp = last->next;
        do {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != last->next);
        cout << endl;
    }
};

int main() {
    CircularLinkedList cll;
    cll.insert(10);
    cll.insert(20);
    cll.insert(30);
    cll.display();
    return 0;
}

```

Output: 10 20 30

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Implement a program of binary search tree (BST) operations.

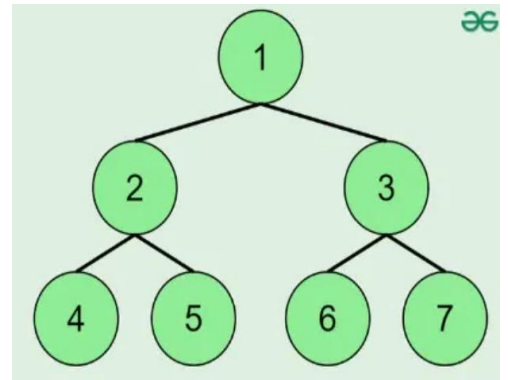
Binary search tree is a data structure that quickly allows us to maintain a sorted list of numbers.

It is called a binary tree because each tree node has a maximum of two children.

It is called a search tree because it can be used to search for the presence of a number in $O(\log(n))$ time.

The properties that separate a binary search tree from a regular binary tree is:

1. All nodes of left subtree are less than the root node
2. All nodes of right subtree are more than the root node
3. Both subtrees of each node are also BSTs i.e. they have the above two properties



Algorithm

1. Start
2. Set root = NULL
3. Insert 50: since root is NULL, create node and make it root
4. Insert 30: compare with root, go left and insert
5. Insert 70: compare with root, go right and insert
6. Insert remaining values (20, 40, 60, 80) by comparing and placing them
7. Perform inorder traversal:
 - a. Traverse left subtree
 - b. Visit the node
 - c. Traverse right subtree
8. Read the key to search from user
9. Search the key in BST:
 - a. If current node is NULL → not found
 - b. If key equals node data → found
 - c. If key is smaller → search in left subtree
 - d. If key is larger → search in right subtree
10. Display whether the key is found or not
11. Stop

Code:

```
#include <iostream>
using namespace std;
// Node structure
class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int value) {
        data = value;
        left = right = NULL;
    }
};
// Insert into BST
Node* insert(Node* root, int value) {
    if (root == NULL) {
        return new Node(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else {
        root->right = insert(root->right, value);
    }
    return root;
}
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
// Search in BST
bool search(Node* root, int key) {
    if (root == NULL) return false;
    if (root->data == key) return true;
    else if (key < root->data) return search(root->left);
    else return search(root->right);
}
int main() {
    Node* root = NULL;
    // Insert values
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
```

```
insert(root, 20);
insert(root, 40);
insert(root, 60);
insert(root, 80);
cout << "Inorder Traversal: ";
inorder(root);
int key;
cout << "\nEnter value to search: ";
cin >> key;
if (search(root, key))
    cout << "Found!";
else
    cout << "Not Found!";

return 0;
}
```

Output:

Inorder Traversal: 20 30 40 50 60 70 80

Enter value to search: 40

Found!

Name: _____ Class: _____
Expt.Title: _____
Roll No: _____ Expt No: _____ Date: _____
Remarks: _____ Sign: _____

Title: Write and demonstrate programs for in-order, pre-order, and post-order traversal of trees.

1) **In-order Traversal:** visits nodes in the order: “Left → Root → Right”

In Binary Search Tree (BST), it gives sorted output.

INORDER(node)

1. If node is NULL, return
2. INORDER(node->left)
3. Visit (print) node->data
4. INORDER(node->right)

2) **Pre-order Traversal:** visits nodes in the order: “Root → Left → Right”

It is useful for copying the tree or creating prefix expressions.

PREORDER(node)

1. If node is NULL, return
2. Visit (print) node->data
3. PREORDER(node->left)
4. PREORDER(node->right)

3). **Post-order Traversal:** visits nodes in the order: “Left → Right → Root”

It is used when deleting a tree because children are visited before the parent.

POSTORDER(node)

1. If node is NULL, return
2. POSTORDER(node->left)
3. POSTORDER(node->right)
4. Visit (print) node->data

Code:

```
#include <iostream>
using namespace std;
// Define a Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    // Constructor to initialize a new node
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};
// Function for In-order traversal (Left -> Root -> Right)
void inorderTraversal(Node* node) {
    if (node == nullptr) {
        return;
    }
    inorderTraversal(node->left); // Traverse left subtree
```

```

    cout << node->data << " "; // Visit root
    inorderTraversal(node->right); // Traverse right subtree
}
// Function for Pre-order traversal (Root -> Left -> Right)
void preorderTraversal(Node* node) {
    if (node == nullptr) {
        return;
    }
    cout << node->data << " "; // Visit root
    preorderTraversal(node->left); // Traverse left subtree
    preorderTraversal(node->right); // Traverse right subtree
}
// Function for Post-order traversal (Left -> Right -> Root)
void postorderTraversal(Node* node) {
    if (node == nullptr) {
        return;
    }
    postorderTraversal(node->left); // Traverse left subtree
    postorderTraversal(node->right); // Traverse right subtree
    cout << node->data << " "; // Visit root
}
int main() {
    // Create a sample binary tree
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->left = new Node(6);
    root->right->right = new Node(7);
    cout << "In-order Traversal: ";
    inorderTraversal(root);
    cout << std::endl;
    cout << "Pre-order Traversal: ";
    preorderTraversal(root);
    cout << endl;
    cout << "Post-order Traversal: ";
    postorderTraversal(root);
    cout << endl;
    return 0;
}

```

Output:

In-order Traversal: 4 2 5 1 6 3 7
 Pre-order Traversal: 1 2 4 5 3 6 7
 Post-order Traversal: 4 5 2 6 7 3 1

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Discover the program for creation and operations on a heap tree.

A Heap is a special type of Complete Binary Tree in which all levels are completely filled except possibly the last level, and nodes are filled from left to right. Heaps are mainly of two types: Max Heap and Min Heap. In a Max Heap, the value of each parent node is greater than or equal to the values of its children, so the largest element is always stored at the root. In a Min Heap, each parent node is smaller than or equal to its children, keeping the smallest element at the root.

Algorithm for Max Heap Insertion

1. Start
2. Insert the new element at the end of the heap
3. Compare the inserted element with its parent
4. If the parent is smaller, swap both
5. Repeat step 3 until the heap property is restored
6. Stop

Algorithm for Max Heap Deletion

1. Start
2. Replace the root element with the last element
3. Reduce heap size by 1
4. Compare the root with its children
5. If any child is larger, swap root with the larger child
6. Repeat step 4 until heap property is restored
7. Stop

Code:

```
#include <iostream>
using namespace std;
class MaxHeap {
private:
    int arr[50]; // Fixed-size heap array
    int size;    // Number of elements in heap
public:
    MaxHeap() {
        size = 0; // Initially heap is empty
    }
    // Function to insert (add) element in heap
    void insert(int value) {
        size++;
        arr[size] = value; // Add at last position
        int i = size;
        // Fix the heap by percolating up
```

```

while (i > 1 && arr[i/2] < arr[i]) {
    swap(arr[i], arr[i/2]);
    i = i / 2;
}
cout << value << " inserted.\n";
}
// Function to delete maximum element (root)
void deleteMax() {
    if (size == 0) {
        cout << "Heap is empty!\n";
        return;
    }
    cout << "Deleted max element: " << arr[1] << endl;
    arr[1] = arr[size]; // Move last element to root
    size--;
    int i = 1;
    // Percolate down
    while (true) {
        int left = i * 2;
        int right = i * 2 + 1;
        int largest = i;
        if (left <= size && arr[left] > arr[largest])
            largest = left;
        if (right <= size && arr[right] > arr[largest])
            largest = right;
        if (largest != i) {
            swap(arr[i], arr[largest]);
            i = largest;
        } else {
            break;
        }
    }
}
// Display heap (level-order)
void display() {
    if (size == 0) {
        cout << "Heap is empty.\n";
        return;
    }
    cout << "Heap elements: ";
    for (int i = 1; i <= size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
};
int main() {
    MaxHeap heap;
    heap.insert(50);
    heap.insert(30);
    heap.insert(40);
    heap.insert(10);
    heap.insert(60);

```

```
    heap.display();  
    heap.deleteMax();  
    heap.display();  
    return 0;  
}
```

Output:

50 inserted.

30 inserted.

40 inserted.

10 inserted.

60 inserted.

Heap elements: 60 50 40 10 30

Deleted max element: 60

Heap elements: 50 30 40 10

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Design and implement depth-first traversal (DFS) & Breadth-First- Search(BFS) for graphs.

- 1. DFS (Depth First Search) :** DFS explores a graph by moving deep along one path before backtracking, using recursion or a stack for traversal. It is useful for path finding, cycle detection, and maze-solving applications.
- 2. BFS (Breadth First Search) :** BFS visits nodes level by level starting from the nearest nodes first, using a queue for traversal. It is useful for finding the shortest path in unweighted graphs and for network broadcasting.

Algorithm for BFS

1. Start
2. Select a starting node and mark it as visited
3. Insert the starting node into a queue
4. While the queue is not empty:
 - a. Remove a node from the queue
 - b. Visit all its unvisited adjacent nodes
 - c. Mark each adjacent node as visited and insert them into the queue
5. Stop

Algorithm for DFS

1. Start
2. Select a starting node and mark it as visited
3. Visit the starting node
4. For each unvisited adjacent node:
 - a. Recursively apply DFS on that node
5. Stop

Code:

```
#include <iostream>
#include <queue>
using namespace std;
int A[10][10];    // adjacency matrix
int visited[10];  // visited array
int n;            // number of nodes

// ----- DFS -----
void DFS(int v) {
    visited[v] = 1;
    cout << v << " ";
    for (int i = 1; i <= n; i++) {
        if (A[v][i] == 1 && visited[i] == 0) {
            DFS(i);
        }
    }
}
```

```

    }
}
}
// ----- BFS -----
void BFS(int start) {
    queue<int> q;
    for (int i = 1; i <= n; i++)
        visited[i] = 0;    // reset visited

    visited[start] = 1;
    q.push(start);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        cout << v << " ";
        for (int i = 1; i <= n; i++) {
            if (A[v][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                q.push(i);
            }
        }
    }
}
}
int main() {
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "\nEnter adjacency matrix:\n";
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> A[i][j];
        }
    }
    cout << "\nDFS Traversal (starting from 1): ";
    for (int i = 1; i <= n; i++) visited[i] = 0; // reset
    DFS(1);
    cout << "\nBFS Traversal (starting from 1): ";
    BFS(1);
    return 0;
}

```

Output:

Enter number of nodes: 4

Enter adjacency matrix:

0 1 0 1

1 0 1 0

0 1 0 1

1 0 1 0

DFS Traversal (starting from 1): 1 2 3 4

BFS Traversal (starting from 1): 1 2 4 3

Name: _____ Class: _____
Expt.Title: _____
Roll No: _____ Expt No: _____ Date: _____
Remarks: _____ Sign: _____

Title: Illustrate the program of Dijkstra's algorithm to find the shortest path in a graph.

Dijkstra's algorithm is used to find the shortest path from a source node to all other nodes in a weighted graph with non-negative weights. It works by selecting the minimum distance vertex, fixing its shortest path, and updating distances of its neighbors. It uses a greedy strategy and ensures the shortest path is found.

Algorithm:

1. Start
2. Initialize distance[] for all vertices as infinity and set distance of source vertex = 0
3. Mark all vertices as unvisited
4. Repeat until all vertices are visited:
 - a. Select the unvisited vertex with the smallest distance value
 - b. Mark this vertex as visited
 - c. For each adjacent vertex:
 - i. Calculate new distance = current distance + edge weight
 - ii. If new distance is smaller than the stored distance, update the distance[]
5. Print the shortest distance from source to all vertices
6. Stop

Code:

```
#include <iostream>
using namespace std;
#define INF 9999 // Infinity value
#define V 5 // Number of vertices in the graph
// Function to find the vertex with minimum distance
int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex;
    for (int v = 0; v < V; v++) {
        if (visited[v] == false && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}
// Dijkstra Algorithm Function
void dijkstra(int graph[V][V], int src) {
    int dist[V]; // Distance from source
```



```

bool visited[V]; // True when shortest path is fixed
// Initialize distances
for (int i = 0; i < V; i++) {
    dist[i] = INF;
    visited[i] = false;
}
dist[src] = 0; // Source distance = 0
// Find shortest path for V-1 vertices
for (int count = 0; count < V - 1; count++) {
    int u = minDistance(dist, visited);
    visited[u] = true
    for (int v = 0; v < V; v++) {

        if (!visited[v] && graph[u][v] &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}
// Print Result
cout << "Vertex  Distance from Source\n";
for (int i = 0; i < V; i++)
    cout << i << "\t\t" << dist[i] << endl;
}
int main() {
    // Weighted Graph (Adjacency matrix)
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    dijkstra(graph, 0); // Start from vertex 0
    return 0;
}

```

Output:

Vertex	Distance from Source
0	0
1	2
2	5
3	6
4	7

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Illustrate the program of Floyd-Warshall's algorithm to compute all-pair shortest paths.

The Floyd-Warshall algorithm is used to find shortest paths between every pair of vertices in a weighted graph. It uses a dynamic programming approach and repeatedly checks if going through an intermediate vertex gives a shorter path. The algorithm updates the distance matrix for all vertices one by one. It works with positive or negative weights (but no negative cycles).

Algorithm:

1. Start
2. Create a distance matrix $dist[][]$ where $dist[i][j]$ represents the cost of the edge from vertex i to j (or infinity if no direct edge exists)
3. For each vertex k from 1 to n :
 - a. For each vertex i from 1 to n :
 - i. For each vertex j from 1 to n :
 - Update $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$
4. After all iterations, $dist[][]$ contains the shortest distance between every pair of vertices
5. Stop

Code:

```
#include <iostream>
using namespace std;
#define V4
#define INF 9999 // Infinity (very large number)
void floydWarshall(int graph[V][V]) {
    int dist[V][V];
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            dist[i][j] = graph[i][j];
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}
cout << "All-Pairs Shortest Path Matrix:\n";
for (int i = 0; i < V; i++)
```

```

    {
    for (int j = 0; j < V; j++) {
        if (dist[i][j] == INF)
            cout << "INF ";
        else
            cout << dist[i][j] << " ";
        }
    cout << endl;
    }
}

int main() {
    int graph[V][V] = {
        {0, 3, INF, 5},
        {2, 0, INF, 4},
        {INF, 1, 0, INF},
        {INF, INF, 2, 0}
    };
    floydWarshall(graph);
    return 0;
}

```

Output:

All-Pairs Shortest Path Matrix:

```

0 3 7 5
2 0 6 4
3 1 0 5
5 3 2 0

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop a program to construct a minimum spanning tree using Prim's algorithm.

A Minimum Spanning Tree (MST) is a subset of edges of a connected weighted graph that connects all the vertices with the minimum total cost and without forming any cycles. Prim's algorithm builds the MST by starting from any one vertex and repeatedly selecting the smallest-weight edge that connects a visited node to an unvisited node. It grows the spanning tree one edge at a time, always choosing the minimum cost edge that maintains connectivity.

Algorithm:

1. Start
2. Select any vertex as the starting vertex and include it in the MST
3. Mark this vertex as visited
4. Repeat until all vertices are included:
 - a. From all the edges that connect visited and unvisited vertices, choose the edge with the minimum weight
 - b. Add this minimum-weight edge to the MST
 - c. Mark the connected unvisited vertex as visited
5. Display the edges included in the MST
6. Stop

Code:

```
#include <iostream>
#include <limits.h>
using namespace std;
int COST[7][7] = {
    {0,0,0,0,0,0,0},
    {0,9999,10,9999,30,45,9999},
    {0,10,9999,50,9999,40,25},
    {0,9999,50,9999,9999,35,15},
    {0,30,9999,9999,9999,9999,20},
    {0,45,40,35,9999,9999,55},
    {0,9999,25,15,20,55,9999}
};
class GRAPH {
private:
    int n, NEAR[10], T[10][3], mincost;
public:
    GRAPH(int);
    void SHOW_GRAPH();
```

```

    void PRIMS();
};
GRAPH::GRAPH(int par) {
    n = par;
}
void GRAPH::SHOW_GRAPH() {
    cout << "\nAdjacency Matrix:\n";
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++)
            cout << COST[i][j] << "\t";
        cout << endl;
    }
}
void GRAPH::PRIMS() {
    int i, j, k=1, l=1;

    int min = 9999;
    // Step 1: Find smallest edge
    for(i = 1; i <= n; i++) {
        for(j = i; j <= n; j++) {
            if(COST[i][j] < min) {
                min = COST[i][j];
                k = i;
                l = j;
            }
        }
    }
    mincost = COST[k][l];
    T[1][1] = k;
    T[1][2] = l;
    // Step 2: Initialize NEAR array
    for(i = 1; i <= n; i++) {
        if(COST[i][k] < COST[i][l])
            NEAR[i] = k;
        else
            NEAR[i] = l;
    }
    NEAR[k] = 0;
    NEAR[l] = 0;
    // Step 3: Select remaining n-2 edges
    for(i = 2; i <= n-1; i++) {
        min = 9999;
        for(j = 1; j <= n; j++) {
            if(NEAR[j] != 0 && COST[j][NEAR[j]] < min) {
                min = COST[j][NEAR[j]];
                k = j;
            }
        }
        T[i][1] = k;
        T[i][2] = NEAR[k];
        mincost += COST[k][NEAR[k]];
        NEAR[k] = 0;
    }
}

```

```

// Update NEAR array
for(j = 1; j <= n; j++) {
    if(NEAR[j] != 0 && COST[j][k] < COST[j][NEAR[j]])
        NEAR[j] = k;
}
}
// Step 4: Display Output
if(mincost >= 9999) {
    cout << "\nNo Spanning Tree Exists!";
} else {
    cout << "\nMinimum Spanning Tree Edges:\n";
    for(i = 1; i < n; i++)
        cout << T[i][1] << " -- " << T[i][2] << endl;

    cout << "\nCost of Minimum Spanning Tree = " << mincost << endl;
}
}
int main() {
    int n;
    cout << "Enter number of nodes (max 6): ";
    cin >> n;
    GRAPH obj(n);
    obj.PRIMS();
    obj.SHOW_GRAPH();
    return 0;
}

```

Output:

Enter number of nodes (max 6): 6

Minimum Spanning Tree Edges:

1 -- 2

6 -- 2

3 -- 6

4 -- 6

5 -- 3

Cost of Minimum Spanning Tree = 105

Adjacency Matrix:

9999	10	9999	30	45	9999
10	9999	50	9999	40	25
9999	50	9999	9999	35	15
30	9999	9999	9999	9999	20
45	40	35	9999	9999	55
9999	25	15	20	55	9999

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop a program to construct a minimum spanning tree using Kruskal's algorithm.

Kruskal's algorithm is a greedy method used to find the Minimum Spanning Tree (MST) of a connected, weighted graph. It works by sorting all edges in increasing order of weight and then adding them one by one to the MST without creating a cycle. To check cycles, it uses the Union-Find (Disjoint Set) technique. The algorithm stops when exactly $(n - 1)$ edges are included. It always produces the MST with minimum total cost.

Algorithm:

Sort all edges of the graph in non-decreasing order of their weights.

1. Initialize an empty set MST.
2. For each edge (u, v) in sorted order:
 - If adding the edge does not create a cycle, include it in MST (checked using Union-Find).
3. Stop when the MST contains $V - 1$ edges.
4. Print the selected MST edges and their total cost.

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;
struct Edge {
    int u, v, w;
};
// Compare edges by weight
bool cmp(Edge a, Edge b) {
    return a.w < b.w;
}
int parent[50];
// Find parent (Union-Find)
int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}
// Union
void unionSet(int a, int b) {
    parent[find(a)] = find(b);
}
int main() {
    int n, m;
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter number of edges: ";
```

```

cin >> m;
Edge edges[m];
cout << "Enter edges (u v weight):\n";
for (int i = 0; i < m; i++) {
    cin >> edges[i].u >> edges[i].v >> edges[i].w;
}
// Sort edges by weight
sort(edges, edges + m, cmp);
// Initialize parent array
for (int i = 1; i <= n; i++) parent[i] = i;
int mstCost = 0;
cout << "\nEdges in Minimum Spanning Tree:\n";
int count = 0;
for (int i = 0; i < m && count < n - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    if (find(u) != find(v)) {
        cout << u << " -- " << v << " (weight: " << edges[i].w << ")\n";
        mstCost += edges[i].w;
        unionSet(u, v);
        count++;
    }
}
cout << "\nTotal Minimum Cost = " << mstCost << endl;
return 0;
}

```

Output:

Enter number of vertices: 4

Enter number of edges: 5

Enter edges (u v weight):

1 2 10

1 3 6

2 3 5

2 4 15

3 4 4

Edges in Minimum Spanning Tree:

3 -- 4 (weight: 4)

2 -- 3 (weight: 5)

1 -- 3 (weight: 6)

Total Minimum Cost = 15

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Illustrate a program to implement a hash table using appropriate hash functions.

A Hash Table is a data structure that stores data in key–value pairs using a hash function. The hash function converts a key into an array index, allowing fast insertion, search, and deletion in $O(1)$ average time. When two keys map to the same index (collision), techniques like linear probing or chaining are used. Hash tables are widely used in databases, compilers, and symbol tables

Algorithm:

1. Create an array `hashTable[size]` initialized with -1.
2. For each key, compute `index = key % size`.
3. If the index is empty, insert the key.
4. If occupied, move sequentially (`index+1`) until an empty slot is found.
5. For searching, repeat the same probing until the key is found or an empty slot is reached.
6. Display the hash table.

Code:

```
#include <iostream>
using namespace std;
class HashTable {
public:
    int size;
    int *table;
    HashTable(int s) {
        size = s;
        table = new int[size];
        for (int i = 0; i < size; i++)
            table[i] = -1; // -1 means EMPTY
    }
    // Hash function
    int hashFunction(int key) {
        return key % size;
    }
    // Insert using linear probing
    void insert(int key) {
        int index = hashFunction(key);
        while (table[index] != -1) {
            index = (index + 1) % size; // move to next index
        }
        table[index] = key;
    }
    // Search key
```

```

bool search(int key) {
    int index = hashFunction(key);
    int start = index;
    while (table[index] != -1) {
        if (table[index] == key)

return true;
        index = (index + 1) % size;
        if (index == start)
            return false;
    }
    return false;
}
// Display hash table
void display() {
    cout << "\nHash Table:\n";
    for (int i = 0; i < size; i++)
        cout << i << " -> " << table[i] << endl;
}
};
int main() {
    int n;
    cout << "Enter hash table size: ";
    cin >> n;
    HashTable ht(n);
    int keys[] = {12, 25, 35, 26, 14};
    int count = 5;
    cout << "\nInserting keys: 12, 25, 35, 26, 14\n";
    for (int i = 0; i < count; i++)
        ht.insert(keys[i]);
    ht.display();
    int searchKey;
    cout << "\nEnter a key to search: ";
    cin >> searchKey;
    if (ht.search(searchKey))
        cout << "Key FOUND in hash table.\n";
    else
        cout << "Key NOT found.\n";
    return 0;
}

```

Output:

```

Enter hash table size: 6
Inserting keys: 12, 25, 35, 26, 14
Hash Table:
0 -> 12
1 -> 25
2 -> 26
3 -> 14
4 -> -1
5 -> 35

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop a program to perform linear search.

Linear search process starts comparing search element with the first element in the list. If both are matched then result is element found otherwise search element is compared with the next element in the list. Repeat the same until search element is compared with the last element in the list, if that last element also doesn't match, then the result is "Element not found in the list". That means, the search element is compared with element by element in the list.

Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit

Code:

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    int key = 30; // Element to search
    bool found = false;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            cout << "Element " << key << " found at position " << i << endl;
            found = true;
            break;
        }
    }
    if (!found)
        cout << "Element " << key << " not found" << endl;

    return 0;
}
```

Output:

Element 30 found at position 2

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Develop a program to perform Binary search.

Binary search is used only with a list of elements that are already arranged in an order. The binary search can not be used for a list of elements arranged in random order. This search process starts comparing the search element with the middle element in the list. If both are matched, then the result is "element found". Otherwise, we check whether the search element is smaller or larger than the middle element in the list. If the search element is smaller, then we repeat the same process for the left sublist of the middle element. If the search element is larger, then we repeat the same process for the right sublist of the middle element.

Algorithm

Binary_Search (a, lower_bound, upper_bound, val)

Where, 'a' is the given array,

'lower_bound' is the index of the first array element,

'upper_bound' is the index of the last array element,

'val' is the value to search

Step 1: set beg = lower_bound, end = upper_bound, pos = - 1

Step 2: repeat steps 3 and 4 while beg <=end

Step 3: set mid = (beg + end)/2

Step 4: if a[mid] = val

 set pos = mid

 print pos

 go to step 6

 else if a[mid] > val

 set end = mid - 1

 else

 set beg = mid + 1

 [end of if]

 [end of loop]

Step 5: if pos = -1

 print "value is not present in the array"[end of if]

Step 6: exit

Code:

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    int key = 30; // Element to search

    int low = 0, high = n - 1, mid;
    bool found = false;

    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key) {
            cout << "Element " << key << " found at position " << mid << endl;
            found = true;
            break;
        }
        else if (arr[mid] < key)
        {
            low = mid + 1; // Search right half
        }
        else {
            high = mid - 1; // Search left half
        }
    }
    if (!found)
        cout << "Element " << key << " not found" << endl;

    return 0;
}
```

Output : Element 30 found at position 2

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program to implement bubble sort.

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

Algorithm :

```
begin BubbleSort(arr)
  for all array elements
    if arr[i] > arr[i+1]
      swap(arr[i], arr[i+1])
    end if
  end for
  return arr
end BubbleSort
```

Code:

```
#include <iostream>
using namespace std;
int main() {
  int arr[] = {40, 10, 50, 20, 30}; // fixed array
  int n = 5;
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        swap(arr[j], arr[j + 1]);
      }
    }
  }
  cout << "Sorted array: ";
  for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
  return 0;
}
```

Output: Sorted array: 10 20 30 40 50

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program to implement selection sort.

In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. In this algorithm, the array is divided into two parts, first is sorted part, and another one is the unsorted part. Initially, the sorted part of the array is empty, and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

Algorithm:

- Step 1. Start
- Step 2. Create an array & with n elements.
- Step 3. Let i=0
- Step 4. Repeat steps 5 to 10 as long as i<n-1
- Step 5. Let smallest = i
- Step 6. Let j=i+1 Repeat steps 7 to 8 as long as j<n
- Step 7. If $x[j] < x[\text{smallest}]$ then smallest = j. Increment j by 1.
- Step 8. Interchange $x[i]$ & $x[\text{smallest}]$
- Step 9. Increment i by 1.
- Step 10. Stop

Code:

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = 5;
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swap(arr[i], arr[minIndex]);
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Output: Sorted array: 11 12 22 25 64

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program to implement insertion sort.

Insertion sort algorithm arranges a list of elements in a particular order. In insertion sort algorithm, every iteration moves an element from unsorted portion to sorted portion until all the elements are sorted in the list.

Algorithm

Step 1: For i = 1 to n-1
Step 2: Set temp = Array[i]
Step 3: Set j = i - 1
Step 4: while temp <= Array[j]
 Set Array[j + 1] = Array[j]
 Set j = j - 1
 [end of step 4]
Step 5: Set Array[j + 1] = temp
 [end of step 1]
Step 6: stop

Code:

```
#include<iostream>
using namespace std;
void INSERTION_SORT(int arr[], int n)
{
    for(int i=1;i<=n;i++)
    {
        int ele=arr[i];
        for(int j=i-1;(j>=1) && (ele<arr[j]);j--)
        {
            arr[j+1]=arr[j];arr[j]=ele;
        }
    }
}
int main()
{
    int n,arr[10];
    cout<<"**Enter size of an array**"<<endl;
    cin>>n;
    cout<<"\nEnter elements in array"<<endl;
    for(int i=1;i<=n;i++)
    {
        cin>>arr[i];
    }
}
```



```
INSERTION_SORT(arr,n);
cout<<"\nSorted elements are:"<<endl;
for(int i=1;i<=n;i++)
{
cout<<arr[i]<<"\t";
}
return 0;
}
```

Output:

****Enter size of an array** : 5**

Enter elements in array

12

3

5

45

2

Sorted elements are:

2 3 5 12 45

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Apply quick sort technique to sort elements.

Quick sort is a fast sorting algorithm used to sort a list of elements. The quick sort algorithm attempts to separate the list of elements into two parts and then sort each part recursively. That means it uses divide and conquer strategy. In quick sort, the partition of the list is performed based on the element called *pivot*. Here pivot element is one of the elements in the list.

In quick sort, the list is divided into two partitions such that "all elements to the left of pivot are smaller than the pivot and all elements to the right of pivot are greater than or equal to the pivot".

Choosing the pivot

- Step 1 - Pivot can be random, i.e. select the random pivot from the given array.
- Step 2 - Pivot can either be the rightmost element or the leftmost element of the given array.
- Step 3 - Select median as the pivot element.

Algorithm:

Step 1: Initialize low = lb, high = ub
Step 2: Initialize pivot = A [lb]
Step 3: Repeat step 4 to 7 till low < high
Step 4: While (A [low] <= pivot && low < high)
 low++
Step 5: while (A [high] > pivot)
 high--
Step 6: If (low < high)
 Interchange A [low] & A [high]
Step 7: Else interchange A [high] & pivot
 j = high that is pivot position = high.

Code:

```
#include<iostream>
using namespace std;
void QUICK_SORT(int arr[], int low, int high)
{
    int pivot,i,j,temp;
    if(low<high)
    {
        pivot=arr[low];
        i=low+1;
        j=high;
        while(i< j)
```

```

        {
            while(arr[i]<pivot && i<=high)
                i++;
            while(arr[j]>pivot && j>=low)
                j--;
            if(i<j)
            {
                temp=arr[i];
                arr[i]=arr[j];
                arr[j]=temp;
            }
            arr[low]= arr[j];
            arr[j]=pivot;
            QUICK_SORT(arr,low,j-1);
            QUICK_SORT(arr,j+1,high);
        }
    }
}

int main()
{
    int n,arr[10];
    cout<<"Enter size of an array"<<endl;
    cin>>n;
    cout<<"*****Enter elements in an array*****"<<endl;
    for(int i=1;i<=n;i++)
    {
        cin>>arr[i];
    }
    QUICK_SORT(arr,1,n);
    cout<<"Array after sort"<<endl;
    for(int i=1;i<=n;i++)
    {
        cout<<arr[i]<<" ";
    }
    return 0;
}

```

Output:

```

Enter size of an array
4
*****Enter elements in an array*****
12
26
4
3

Array after sort
3 4 12 26

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program to implement Merge sort.

Merge Sort is an efficient sorting technique based on the **Divide and Conquer** strategy. In this method, the given array is repeatedly divided into two halves until each sub-array contains only one element. Since a single element is already sorted, these small sorted parts are then merged in a systematic way to form larger sorted sub-arrays. Finally, all sub-arrays are merged together to produce the fully sorted array.

How Merge Sort Works:

1. **Divide:** Split the array into two halves.
2. **Conquer:** Recursively sort the left half and the right half.
3. **Combine:** Merge the two sorted halves to form a single sorted array

Algorithm:

1. Start
2. If the array has 1 or 0 elements, it is already sorted → Return
3. Divide the array into two halves: left half and right half
4. Recursively apply Merge Sort on the left half
5. Recursively apply Merge Sort on the right half
6. Merge the two sorted halves:
 - a. Compare elements of both halves
 - b. Copy the smaller element into the temporary array
 - c. Continue until all elements are merged
7. Copy the merged elements back to the original array
8. Stop

Code:

```
#include <iostream>
using namespace std;
int arr[100]; // global array
void mergeArray(int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    int brr[100];
    // Merge two sorted halves
    while (i <= mid && j <= high) {
        if (arr[i] < arr[j]) {
            brr[k] = arr[i];
            i++;
        } else {
            brr[k] = arr[j];
            j++;
        }
        k++;
    }
}
```

```

// Copy remaining elements from left half
while (i <= mid) {
    brr[k] = arr[i];
    i++;
    k++;
}

// Copy remaining elements from right half
while (j <= high) {
    brr[k] = arr[j];
    j++;
    k++;
}

// Copy back to original array
for (k = low; k <= high; k++) {
    arr[k] = brr[k];
}
}

void mergeSort(int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;

        mergeSort(low, mid);    // Left half
        mergeSort(mid + 1, high); // Right half
        mergeArray(low, mid, high);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    mergeSort(0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << "\t";
    }
    return 0;
}

```

Output:

```

Enter size of array: 5
Enter elements: 12
36
7
2
0
Sorted array: 0    27    12    36

```

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Write a program to implement radix sort.

Radix Sort is a non-comparative sorting algorithm that sorts numbers digit by digit. It processes each digit starting from the least significant digit (LSD) to the most significant digit. For each digit position, a stable sorting method like Counting Sort is applied. Finally, after processing all digits, the array becomes fully sorted.

Algorithm:

1. Start
2. Find the maximum number to know the number of digits
3. For each digit place (1s, 10s, 100s,...):
 - a. Perform Counting Sort based on that digit
4. Repeat until all digit places are processed
5. Stop

Code:

```
#include <iostream>
using namespace std;
// Function to get maximum element from array
int getMax(int arr[], int n) {
    int maxi = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxi)
            maxi = arr[i];
    }
    return maxi;
}
// Counting sort for a particular digit (exp = 1, 10, 100...)
void countingSort(int arr[], int n, int exp) {
    int output[100];
    int count[10] = {0};
    // Count occurrences of digits
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;
    // Convert count into cumulative count
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
```

```

        // Build output array (stable)
        for (int i = n - 1; i >= 0; i--) {
            int digit = (arr[i] / exp) % 10;
            output[count[digit] - 1] = arr[i];
            count[digit]--;
        }
        // Copy back to original array
        for (int i = 0; i < n; i++)
            arr[i] = output[i];
    }
    // Radix Sort
    void radixSort(int arr[], int n) {
        int maxi = getMax(arr, n);
        // Apply counting sort for every digit
        for (int exp = 1; maxi / exp > 0; exp *= 10)
            countingSort(arr, n, exp);
    }
    int main() {
        int n;
        cout << "Enter number of elements: ";
        cin >> n;
        int arr[100];
        cout << "Enter elements: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
        radixSort(arr, n);
        cout << "Sorted array: ";
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";

        return 0;
    }

```

Output:

Enter number of elements: 3

Enter elements: 2

4

0

Sorted array: 0 2 4

Name: _____ Class: _____

Expt.Title: _____

Roll No: _____ Expt No: _____ Date: _____

Remarks: _____ Sign: _____

Title: Illustrate a program to implement a heap sort on array elements.

Heap Sort is a comparison-based sorting technique that uses a binary heap structure. First, the array is converted into a max-heap, where the largest element is at the root. The root element is swapped with the last element of the array. The heap size is reduced, and heapify is applied again to restore the heap. Repeating this process results in a fully sorted array in ascending order.

Algorithm:

1. Start
2. Build a Max-Heap from the given array
3. Repeat until the heap size becomes 1:
 - a. Swap the first element (largest) with the last element of the heap
 - b. Reduce the heap size by 1
 - c. Heapify the root element to restore the Max-Heap property
4. Stop (array is sorted in ascending order)

Code:

```
#include <iostream>
using namespace std;
// To heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;    // root
    int left = 2 * i + 1; // left child
    int right = 2 * i + 2; // right child
    // Check if left child is larger
    if (left < n && arr[left] > arr[largest])
        largest = left;
    // Check if right child is larger
    if (right < n && arr[right] > arr[largest])
        largest = right;
    // Swap and continue heapifying
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```



```

// Function to perform Heap Sort
void heapSort(int arr[], int n) {
    // Step 1: Build max-heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // Step 2: Extract largest element one by one
    for (int i = n - 1; i >= 0; i--) {
        swap(arr[0], arr[i]); // move root to end
        heapify(arr, i, 0); // heapify reduced heap
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    heapSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Output:

Enter number of elements: 5

Enter elements: 12

6

1

44

8

Sorted array: 1 6 8 12 44